

1. Write a C program that accepts a positive integer n from the user and prints the first n natural numbers (1, 2, 3, ... n)
2. Write a C program that accepts a positive integer n from the user, then reads n numbers one by one and prints their sum
3. Write a C program that repeatedly reads numbers from the user and keeps adding them to a running total. The program should stop taking input as soon as the user enters 0, and then display the final sum
4. Write a C program that accepts a positive integer n (the maximum count of numbers to be entered), then reads numbers one by one and adds them to a sum. If the user enters a negative number at any point, the program should immediately stop taking further input and display the sum calculated so far.
5. Write a C program that accepts a positive integer n , then reads n numbers one by one. If a negative number is entered, it should be ignored (not added to the sum), and the program should continue asking for the remaining numbers. Finally, display the total sum of only the positive numbers entered
6. Write a C program that accepts a positive integer n from the user and prints the first n terms of the Fibonacci series
7. Write a C program that accepts a number (or string) from the user and checks whether it is a palindrome or not (i.e., it reads the same forwards and backwards). Display an appropriate message.
8. Write a C program that accepts a number from the user and checks whether it is an Armstrong number or not (a number that equals the sum of its own digits each raised to the power of the number of digits). Display an appropriate message
9. Write a C program that accepts a positive integer n from the user and prints a right-angled triangle pattern using stars (*), with n rows. Row i should contain i stars.

Sample Output (for $n = 5$) is shown below:

```
*
* *
* * *
* * * *
* * * * *
```

10. Write a C program that accepts a positive integer n from the user and prints a right-angled triangle pattern using increasing numbers, with n rows. Row i should contain numbers from 1 to i .

Sample Output (for $n = 5$) is shown below:

```
1
1 2
1 2 3
```

```
1 2 3 4
1 2 3 4 5
```

11. Repeat question 10 with the following change: Row i should contain the number i repeated i times. Sample given below.

```
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
```

12. Write a C program that accepts a positive integer n from the user and prints Pascal's Triangle with n rows. In Pascal's Triangle, each row starts and ends with 1, and every other number is the sum of the two numbers directly above it (from the previous row).

Sample Output:

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
1 5 10 10 5 1
```

13. Write a C program that accepts a number from the user and checks whether it is a prime number or not. Display an appropriate message
14. Write a C program that accepts a positive integer n from the user and prints a hollow square pattern of size $n \times n$ using stars (*) — stars only on the border, and blank spaces inside.

Sample Output:

```
* * * * *
*       *
*       *
*       *
* * * * *
```